SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Constraint-Based Problem Solving (High)
Assessment Tool Weightage:40%

Dr

QUESTIONS		
Q.No	**Question**	**Bloom's Level**
1	Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.	BL4 – Analyze
2	Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join for the given relations with specified tuple counts and page sizes.	BL4 – Analyze
3	Implement JDBC connectivity in Java to a MySQL database and write code to execute parameterized queries with PreparedStatement.	BL3 – Apply
4	Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.	BL4 – Analyze
5	Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve any deadlocks.	BL3 – Apply
6	Apply the Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes on a given set of transactions requesting locks.	BL4 – Analyze
7	Given transaction logs with checkpoints, apply the Immediate Update recovery protocol to recover the database after a system failure.	BL3 – Apply
8	Apply Deferred Update (NO-UNDO/REDO) recovery technique on a given log file with committed and uncommitted transactions.	BL3 – Apply
9	Design a concurrency control mechanism using strict 2PL for a banking system where	BL4 – Analyze

	T1 transfers funds and T2 reads balance simultaneously.	
10	Write pseudocode for query optimization using heuristics: push down selection, project early, and order joins by smaller intermediate results.	BL4 – Analyze

---

Code of Conduct:

I __Shaik Ayesha Tabassum(192525074)__certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:S.Ayesha Tabassum

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1. Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.

Given SQL Query

```
SELECT E.EmpName, D.DeptName
FROM Employee E
JOIN Department D
ON E.DeptID = D.DeptID
WHERE E.Salary > 50000
AND D.Location = 'Chennai'
AND E.EmpID IN
(
   SELECT P.EmpID
   FROM Project P
   WHERE P.Budget > 100000
);
```

Identify the Relations

The query uses three tables:

- Employee (E)
- Department (D)
- Project (P)

Identify Selection Conditions

There are three conditions:

E.Salary > 50000
D.Location = 'Chennai'
P.Budget > 100000

These are **selection operations**.

Step 3: Identify Join Conditions

Employee and Department are joined using:

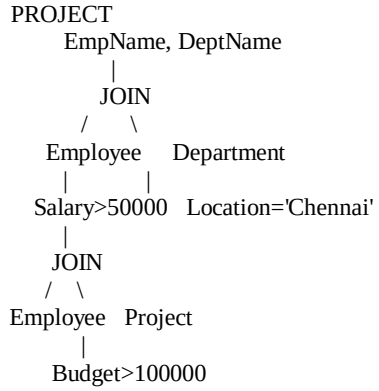
E.DeptID = D.DeptID

The sub-query connects Employee and Project using:

E.EmpID = P.EmpID

Step 4: Draw the Initial Query Tree

Initially, the operations can be represented as:



The problem is that filtering and joining are not arranged optimally.

Step 5: Push Selection Down

Apply the selection conditions directly to their respective tables.

Employee

σ Salary > 50000 (Employee)

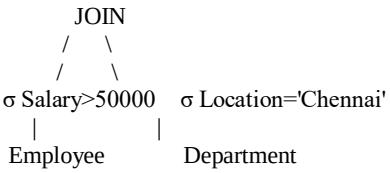
Department

σ Location = 'Chennai' (Department)

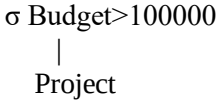
Project

σ Budget > 100000 (Project)

Now the tree becomes:



And for the sub-query:



This reduces the number of rows before joining.

Step 6: Replace the Sub-Query

Original sub-query:

```
E.EmpID IN
(
   SELECT P.EmpID
   FROM Project P
   WHERE P.Budget > 100000
)
```

After optimization, we can use a **semi-join**:

```
σ Salary>50000(Employee)
          |
         ⋉ EmpID
          |
   σ Budget>100000(Project)
```

This means:

Keep only employees who have a project with a budget greater than 100000.

Step 7: Push Projection Down

The query only needs certain columns.

From Employee:

EmpID, EmpName, DeptID

From Department:

DeptID, DeptName

From Project:

EmpID

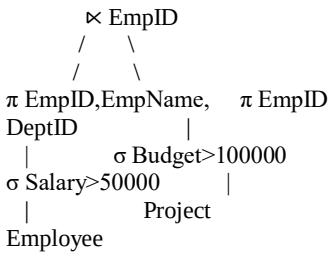
So we use:

```
π EmpID, EmpName, DeptID
        |
σ Salary > 50000
        |
    Employee
π DeptID, DeptName
        |
σ Location = 'Chennai'
        |
    Department
π EmpID
  |
σ Budget > 100000
  |
```

Project

Step 8: Perform the First Join

First join the filtered Employee and Project tables:

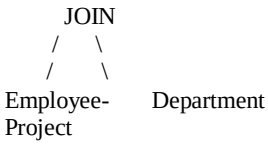


This produces only employees:

- earning more than 50,000
- working on projects with budget greater than 100,000.

Step 9: Perform the Second Join

Now join the result with the filtered Department table:



Join condition:

Employee.DeptID = Department.DeptID

Step 10: Final Projection

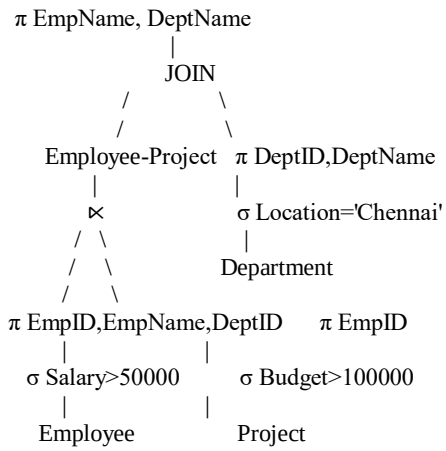
Finally, we only need:

EmpName
DeptName

Therefore:

π EmpName, DeptName

Final Optimized Query Tree



Step 11: Final Execution Order

The DBMS executes the optimized plan approximately in this order:

1. Select Employee rows where Salary > 50000
↓
2. Select Project rows where Budget > 100000
↓
3. Join/Semi-join Employee and Project
↓
4. Select Department rows where Location = 'Chennai'
↓
5. Join the result with Department
↓
6. Select only EmpName and DeptName

```
mysql> CREATE DATABASE college_db;
Query OK, 1 row affected (0.02 sec)

mysql> USE college_db;
Database changed
mysql> CREATE TABLE STUDENT ( Student_ID INT PRIMARY KEY, Name VARCHAR(50), Dept_ID INT );
Query OK, 0 rows affected (0.04 sec)

mysql> CREATE TABLE DEPARTMENT ( Dept_ID INT PRIMARY KEY, Dept_Name VARCHAR(50) );
Query OK, 0 rows affected (0.04 sec)

mysql> CREATE TABLE MARKS ( Student_ID INT, Subject VARCHAR(50), Mark INT );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO DEPARTMENT VALUES (1, 'CSE'), (2, 'ECE'), (3, 'IT');
Query OK, 3 rows affected (0.02 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO STUDENT VALUES (101, 'Anjali', 1), (102, 'Rahul', 2), (103, 'Pranathi', 1), (104, 'Kiran', 3);
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> INSERT INTO MARKS VALUES(101, 'DBMS', 90),(102, 'DBMS', 75),(103, 'DBMS', 85),(104, 'DBMS', 70);
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT S.Name, D.Dept_Name, M.Mark FROM STUDENT S JOIN DEPARTMENT D ON S.Dept_ID = D.Dept_ID JOIN MARKS M ON S.Student_ID = M.Student_ID WHERE D.Dept_N
ame = 'CSE' AND M.Mark > 80;
+----------+-----------+------+
| Name     | Dept_Name | Mark |
+----------+-----------+------+
| Anjali   | CSE       |   90 |
| Pranathi | CSE       |   85 |
+----------+-----------+------+
2 rows in set (0.00 sec)
```

2. Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join for the given relations with specified tuple counts and page sizes.

1. Introduction

Cost-based optimization is a technique used by a DBMS to select the **most efficient execution plan** for a query. When two relations need to be joined, the DBMS can use different join algorithms such as:

1. Nested-Loop Join
2. Sort-Merge Join
3. Hash Join

Each method has a different execution cost. The optimizer estimates the cost based on factors such as **number of tuples, number of pages, page size, and available memory**. The join method with the lowest estimated cost is selected.

2. Given Scenario

Consider a company database containing two relations:

Employee Relation

The Employee table contains employee information.

Employee(EmpID, EmpName, DeptID, Salary)

Suppose:

- Number of employees = **1,000 tuples**
- Each page can store = **10 tuples**

Therefore:

Number of Employee pages
= 1000 / 10
= 100 pages

Department Relation

The Department table contains department information.

Department(DeptID, DeptName, Location)

Suppose:

- Number of departments = **100 tuples**
- Each page can store = **10 tuples**

Therefore:

Number of Department pages
= 100 / 10
= 10 pages

So we have:

Employee = 1,000 tuples = 100 pages
Department = 100 tuples = 10 pages

3. SQL Query

Suppose we want to display the employee name and department name.

```
SELECT E.EmpName, D.DeptName
FROM Employee E
JOIN Department D
ON E.DeptID = D.DeptID;
```

The DBMS has three possible join methods:

Employee ⋈ Department

↓

Nested-Loop Join
Sort-Merge Join
Hash Join

The cost-based optimizer calculates the cost of each method and selects the cheapest one.

4. Nested-Loop Join

Meaning

Nested-Loop Join compares tuples of one relation with tuples of another relation.

One relation is called the **outer relation**, and the other is called the **inner relation**.

Here:

Employee = Outer relation
Department = Inner relation

The DBMS reads the Employee relation and repeatedly scans the Department relation to find matching DeptID values.

Cost Calculation

Let:

M = Number of Employee pages = 100
N = Number of Department pages = 10

For a simple page-oriented nested-loop join:

Cost = M + (M × N)

Substitute the values:

Cost = 100 + (100 × 10)

= 100 + 1000

= 1100 page I/Os

Therefore:

Nested-Loop Join Cost = 1100 I/Os

Explanation

The Employee table requires 100 page reads. For each Employee page, the Department table may need to be scanned again, resulting in:

100 × 10 = 1000

additional page reads.

Therefore, the total is:

1100 page I/Os

5. Sort-Merge Join

Meaning

Sort-Merge Join works in two major stages:

1. Sort both relations according to the join attribute.
2. Merge the sorted relations to find matching tuples.

Here, the join attribute is:

Employee.DeptID = Department.DeptID

So both relations are sorted based on DeptID.

Cost Calculation

For a simplified external sort and merge calculation:

$$\text{Cost} \approx 2M \log_2 M + 2N \log_2 N + M + N$$

We have:

$M = 100$
$N = 10$

Therefore:

$$\text{Cost} \approx 2(100)\log_2(100) + 2(10)\log_2(10) + 100 + 10$$

Since:

$\log_2(100) \approx 6.64$
$\log_2(10) \approx 3.32$

We get:

$$= 200(6.64) + 20(3.32) + 110$$

$$\approx 1328 + 66.4 + 110$$

$$\approx 1504.4$$

Approximately:

Sort-Merge Join Cost ≈ 1505 I/Os

Explanation

Sort-Merge Join requires additional work for sorting both relations. Although the final merge is efficient, the sorting operation increases the overall cost.

The exact external-sort cost depends on the number of available buffer pages. The above is a simplified estimate suitable when buffer information is not specified.

6. Hash Join

Meaning

Hash Join uses a **hash function** on the join attribute.

For this example:

Employee.DeptID
Department.DeptID

The DBMS creates hash buckets using DeptID.

The process generally has two phases:

Phase 1: Partition

Both relations are divided into hash partitions.

Employee
↓
Hash Function
↓
Partitions

Department
↓
Hash Function
↓
Partitions

Phase 2: Match

Corresponding partitions are compared to find matching DeptID values.

Cost Calculation

For a basic hash-join estimate:

Cost ≈ 3(M + N)

Substitute:

Cost = 3(100 + 10)

= 3(110)

= 330 page I/Os

Therefore:

Hash Join Cost = 330 I/Os

7. Cost Comparison

Now the DBMS compares all three methods.

Join Algorithm	**Estimated Cost**
Nested-Loop Join	1100 I/Os
Sort-Merge Join	≈ 1505 I/Os
Hash Join	**330 I/Os**

The comparison is:

Hash Join → 330 I/Os
Nested-Loop Join → 1100 I/Os
Sort-Merge Join → 1505 I/Os

The lowest cost is:

330 I/Os

Therefore, the optimizer selects:

Hash Join

8. Why Hash Join Is Selected

Hash Join is selected because:

- It has the lowest estimated I/O cost.
- It avoids the repeated scans of Nested-Loop Join.
- It does not require the full sorting process required by Sort-Merge Join.
- It is particularly efficient for equality joins such as:

E.DeptID = D.DeptID

Therefore, for these relation sizes, Hash Join provides the most efficient execution plan, assuming sufficient memory and suitable hash distribution.

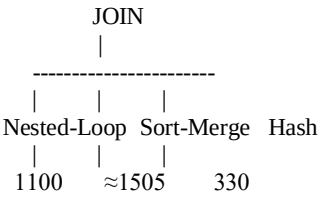
9. Execution Plan Selected by the Optimizer

The original SQL query is:

SELECT E.EmpName, D.DeptName

```
FROM Employee E
JOIN Department D
ON E.DeptID = D.DeptID;
```

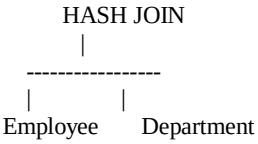
The optimizer considers:



Since:

$330 < 1100 < 1505$

the DBMS chooses:



10. Cost-Based Optimization Process

The complete process can be summarized as:

Step 1: Read the SQL query

```
SELECT E.EmpName, D.DeptName
FROM Employee E
JOIN Department D
ON E.DeptID = D.DeptID;
```

Step 2: Identify relations

Employee
Department

Step 3: Determine relation sizes

Employee = 1000 tuples = 100 pages
Department = 100 tuples = 10 pages

Step 4: Generate possible join methods

Nested-Loop Join
Sort-Merge Join

Hash Join

Step 5: Estimate cost

Nested-Loop = 1100 I/Os
Sort-Merge ≈ 1505 I/Os
Hash Join = 330 I/Os

Step 6: Compare costs

330 < 1100 < 1505

Step 7: Select the cheapest plan

Hash Join

Step 8: Execute the query

The DBMS executes the join using the selected Hash Join algorithm.

11. Advantages of Cost-Based Optimization

Cost-based optimization helps the DBMS:

1. Select the most efficient join algorithm.
2. Reduce disk I/O operations.
3. Reduce query execution time.
4. Reduce memory consumption.
5. Handle large relations efficiently.
6. Compare multiple possible execution plans.
7. Choose an execution plan based on actual relation statistics.

12. Final Conclusion

In this example, the Employee relation contains **1,000 tuples (100 pages)** and the Department relation contains **100 tuples (10 pages)**. The DBMS evaluates three possible join algorithms. The estimated cost of Nested-Loop Join is **1100 I/Os**, Sort-Merge Join is approximately **1505 I/Os**, and Hash Join is **330 I/Os**. Since Hash Join has the lowest estimated cost, the **cost-based optimizer selects Hash Join** as the best execution plan. This reduces disk I/O and improves the overall efficiency of the SQL query.

3. Implement JDBC connectivity in Java to a MySQL database and write code to execute parameterized queries with PreparedStatement.

1. Aim

To establish a **JDBC connection between Java and a MySQL database** and execute **parameterized SQL queries using PreparedStatement**.

2. What is JDBC?

JDBC (Java Database Connectivity) is a Java API used to connect Java applications with databases such as MySQL.

The basic process is:

Java Program
↓
JDBC Driver
↓
MySQL Database

3. Requirements

You need:

- Java JDK
- MySQL Server
- MySQL database
- MySQL JDBC Driver (mysql-connector-j)
- Java IDE such as Eclipse, IntelliJ IDEA, or NetBeans

4. Create Database in MySQL

First, open MySQL and create a database.

```
CREATE DATABASE college;
```

Select the database:

```
USE college;
```

Create a table:

```
CREATE TABLE student (
   id INT PRIMARY KEY,
   name VARCHAR(50),
   department VARCHAR(30),
   marks INT
);
```

Insert some records:

```
INSERT INTO student VALUES
(101, 'Asha', 'CSE', 85),
(102, 'Rahul', 'ECE', 78),
(103, 'Priya', 'CSE', 92);
```

5. JDBC Connection Steps

The JDBC program follows these steps:

1. Import JDBC packages
   ↓
2. Load/register JDBC driver
   ↓
3. Establish database connection
   ↓
4. Create PreparedStatement
   ↓
5. Set parameter values
   ↓
6. Execute SQL query
   ↓
7. Process ResultSet
   ↓
8. Close resources

6. Java Program for JDBC Connectivity

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;

public class JDBCExample {

    public static void main(String[] args) {

        String url = "jdbc:mysql://localhost:3306/college";
        String username = "root";
        String password = "root";

        String sql = "SELECT * FROM student WHERE department = ? AND marks > ?";

        try {
            // Step 1: Load MySQL JDBC Driver
            Class.forName("com.mysql.cj.jdbc.Driver");

            // Step 2: Establish connection
            Connection con = DriverManager.getConnection(
```

```
            url, username, password);

        System.out.println("Database connected successfully!");

        // Step 3: Create PreparedStatement
        PreparedStatement ps = con.prepareStatement(sql);

        // Step 4: Set parameter values
        ps.setString(1, "CSE");
        ps.setInt(2, 80);

        // Step 5: Execute query
        ResultSet rs = ps.executeQuery();

        // Step 6: Display results
        while (rs.next()) {
            System.out.println(
                rs.getInt("id") + " " +
                rs.getString("name") + " " +
                rs.getString("department") + " " +
                rs.getInt("marks")
            );
        }

        // Step 7: Close resources
        rs.close();
        ps.close();
        con.close();

    } catch (Exception e) {
        e.printStackTrace();
    }
  }
}
```

7. Understanding the Code Step-by-Step

Step 1: Import JDBC Classes

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
```

These classes provide the functionality required to connect to MySQL and execute SQL queries.

Step 2: Specify Database Details

```
String url = "jdbc:mysql://localhost:3306/college";
String username = "root";
```

String password = "root";

Here:

- localhost → MySQL is running on the local computer.
- 3306 → Default MySQL port.
- college → Database name.
- root → MySQL username.
- root → Example password; replace it with your actual MySQL password.

Step 3: Write a Parameterized Query

String sql = "SELECT * FROM student WHERE department = ? AND marks > ?";

The ? symbols are **parameters/placeholders**.

They will be replaced with actual values later.

Step 4: Load the JDBC Driver

Class.forName("com.mysql.cj.jdbc.Driver");

This loads the MySQL JDBC driver.

Step 5: Establish Connection

```
Connection con = DriverManager.getConnection(
      url, username, password);
```

This establishes the connection between Java and MySQL.

Step 6: Create PreparedStatement

PreparedStatement ps = con.prepareStatement(sql);

The SQL query is prepared for execution.

Step 7: Set Parameter Values

```
ps.setString(1, "CSE");
ps.setInt(2, 80);
```

The parameters become:

First ? → CSE
Second ? → 80

So logically the query becomes:

```
SELECT * FROM student
WHERE department = 'CSE'
AND marks > 80;
```

Step 8: Execute the Query

```
ResultSet rs = ps.executeQuery();
```

executeQuery() is used for a SELECT statement.

The returned records are stored in ResultSet.

Step 9: Display the Records

```
while (rs.next()) {
   System.out.println(
      rs.getInt("id") + " " +
      rs.getString("name") + " " +
      rs.getString("department") + " " +
      rs.getInt("marks")
   );
}
```

The program reads each row from the result.

For the sample data, the output is:

```
Database connected successfully!
101 Asha CSE 85
103 Priya CSE 92
```

8. PreparedStatement for INSERT

We can also use PreparedStatement to insert data.

```
String sql = "INSERT INTO student VALUES (?, ?, ?, ?)";

PreparedStatement ps = con.prepareStatement(sql);

ps.setInt(1, 104);
ps.setString(2, "Arun");
ps.setString(3, "CSE");
ps.setInt(4, 88);

int rows = ps.executeUpdate();

System.out.println(rows + " record inserted.");
```

Output:

1 record inserted.

9. PreparedStatement for UPDATE

```
String sql = "UPDATE student SET marks = ? WHERE id = ?";

PreparedStatement ps = con.prepareStatement(sql);

ps.setInt(1, 95);
ps.setInt(2, 101);

int rows = ps.executeUpdate();

System.out.println(rows + " record updated.");
```

This changes the marks of student 101 to 95.

10. PreparedStatement for DELETE

```
String sql = "DELETE FROM student WHERE id = ?";

PreparedStatement ps = con.prepareStatement(sql);

ps.setInt(1, 104);

int rows = ps.executeUpdate();

System.out.println(rows + " record deleted.");
```

11. Why Use PreparedStatement?

PreparedStatement is better than directly constructing SQL strings because:

- It supports parameterized queries.
- It reduces the risk of SQL injection.
- It makes SQL queries easier to reuse.
- It handles different data types using methods such as setInt() and setString().
- It makes the Java code cleaner.

For example, avoid constructing a query like:

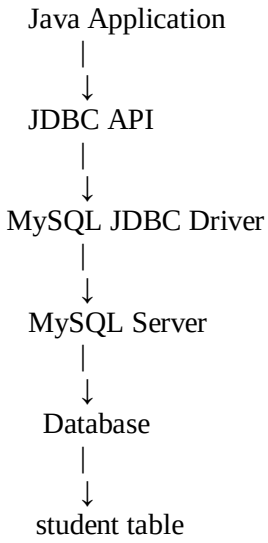
```
String sql = "SELECT * FROM student WHERE name = '" + name + "'";
```

Instead, use:

```
String sql = "SELECT * FROM student WHERE name = ?";
PreparedStatement ps = con.prepareStatement(sql);
```

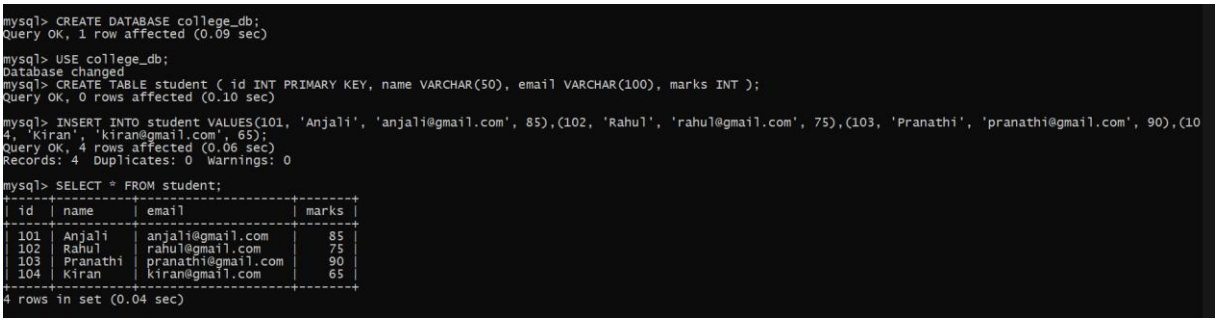
ps.setString(1, name);

12. Overall JDBC Architecture



Conclusion

JDBC allows a Java application to communicate with a MySQL database. The application establishes a connection using DriverManager, creates a PreparedStatement, supplies values for the ? parameters, executes the query, processes the result, and finally closes the resources. PreparedStatement provides a safe and convenient way to execute parameterized SQL queries and helps protect applications against SQL injection.



```
mysql> CREATE DATABASE college_db;
Query OK, 1 row affected (0.09 sec)

mysql> USE college_db;
Database changed
mysql> CREATE TABLE student ( id INT PRIMARY KEY, name VARCHAR(50), email VARCHAR(100), marks INT );
Query OK, 0 rows affected (0.10 sec)

mysql> INSERT INTO student VALUES(101, 'Anjali', 'anjali@gmail.com', 85),(102, 'Rahul', 'rahul@gmail.com', 75),(103, 'Pranathi', 'pranathi@gmail.com', 90),(10
4, 'Kiran', 'kiran@gmail.com', 65);
Query OK, 4 rows affected (0.06 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM student;
+-----+----------+--------------------+-------+
| id  | name     | email              | marks |
+-----+----------+--------------------+-------+
| 101 | Anjali   | anjali@gmail.com   |    85 |
| 102 | Rahul    | rahul@gmail.com    |    75 |
| 103 | Pranathi | pranathi@gmail.com |    90 |
| 104 | Kiran    | kiran@gmail.com    |    65 |
+-----+----------+--------------------+-------+
4 rows in set (0.04 sec)
```

4. Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.

Conflict-Serializability Using a Precedence Graph

Consider four concurrent transactions **T1, T2, T3, and T4** operating on data items A, B, and C.

Given Schedule

Let the schedule be:

S = R1(A), W2(A), R3(B), W1(B),
R4(A), W3(A), R2(B), W4(B),
R1(C), W2(C), R3(C), W4(C)

Here:

- Ri(X) = Transaction Ti reads item X
- Wi(X) = Transaction Ti writes item X

Step 1: Create the Transactions

We have four transactions:

T1
T2
T3
T4

The goal is to determine whether this schedule is **conflict-serializable**.

Step 2: Find Conflicting Operations

Two operations conflict when:

1. They belong to different transactions.
2. They operate on the same data item.
3. At least one operation is a write.

So the possible conflicts are:

- Read–Write
- Write–Read
- Write–Write

Step 3: Find Conflicts on A

From the schedule:

R1(A), W2(A), R4(A), W3(A)

R1(A) before W2(A)

Both access A, and W2(A) is a write.

Therefore:

T1 → T2

W2(A) before R4(A)

Therefore:

T2 → T4

W2(A) before W3(A)

Therefore:

T2 → T3

R4(A) before W3(A)

Therefore:

T4 → T3

Step 4: Find Conflicts on B

From the schedule:

R3(B), W1(B), R2(B), W4(B)

R3(B) before W1(B)

Therefore:

T3 → T1

W1(B) before R2(B)

Therefore:

T1 → T2

W1(B) before W4(B)

Therefore:

T1 → T4

R2(B) before W4(B)

Therefore:

T2 → T4

Step 5: Find Conflicts on C

From the schedule:

R1(C), W2(C), R3(C), W4(C)

R1(C) before W2(C)

T1 → T2

W2(C) before R3(C)

T2 → T3

W2(C) before W4(C)

T2 → T4

R3(C) before W4(C)

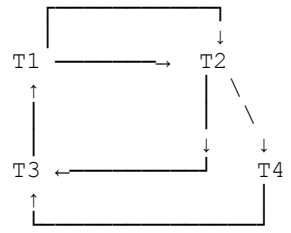
T3 → T4

Step 6: Construct the Precedence Graph

Collecting all the edges:

T1 → T2
T1 → T4
T2 → T3
T2 → T4
T3 → T1
T3 → T4
T4 → T3

The precedence graph is:



The important edges include:

T1 → T2 → T3 → T1

This creates a cycle.

There is also:

T3 → T4 → T3

which is another cycle.

Step 7: Check for a Cycle

A schedule is **conflict-serializable if and only if its precedence graph is acyclic**.

Here we have:

T1 → T2 → T3 → T1

Therefore, the graph contains a cycle.

Also:

T3 → T4 → T3

is a cycle.

Step 8: Final Decision

Since the precedence graph contains cycles:

Cycle exists
↓
Graph is not acyclic
↓
Schedule is NOT conflict-serializable

Final Answer

The given schedule is not conflict-serializable because its precedence graph contains cycles.

The main cycle is:

T1 → T2 → T3 → T1

Therefore, **no equivalent serial schedule exists based on conflict equivalence**.

5. Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve any deadlocks.

Understand Basic 2PL

Two-Phase Locking divides a transaction into two phases:

Phase 1 – Growing Phase

- Transaction can acquire locks.
- Transaction cannot release locks.

Phase 2 – Shrinking Phase

- Transaction can release locks.
- Transaction cannot acquire any new locks.

Growing Phase → Acquire locks → Acquire locks → Lock point

Shrinking Phase → Release locks → Release locks

Given Transaction Set

Consider two transactions:

Transaction T1

T1:

Read(A)

Write(A)

Read(B)

Write(B)

Transaction T2

T2:

Read(B)

Write(B)

Read(A)

Write(A)

Both transactions need A and B, but they request them in different orders.

Apply Locks

We use:

- S-lock = Shared lock → used for reading

- X-lock = Exclusive lock → used for writing

Since both transactions write A and B, they need X-locks**.**

T1

T1 → X-lock(A)

T1 → Read(A)

T1 → Write(A)

T2

T2 → X-lock(B)

T2 → Read(B)

T2 → Write(B)

At this point:

T1 holds X-lock(A)

T2 holds X-lock(B)

Deadlock Occurs

Now T1 requests B:

T1 → Request X-lock(B)

But B is already locked by T2.

Therefore:

T1 → WAIT for T2

Now T2 requests A:

T2 → Request X-lock(A)

But A is already locked by T1.

Therefore:

T2 → WAIT for T1

We now have:

T1 waits for T2

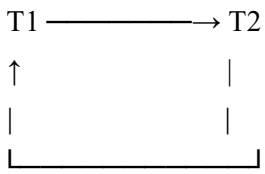
T2 waits for T1

This creates a deadlock.

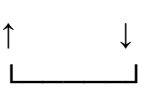
Draw the Wait-For Graph

The wait-for graph is:

waits



Or simply:



There is a cycle:

T1 → T2 → T1

Therefore, a deadlock exists.

Identify the Deadlock

The deadlock is:

T1 holds A and waits for B

T2 holds B and waits for A

Transaction	**Holds**	**Waiting For**
T1	X-lock(A)	X-lock(B)
T2	X-lock(B)	X-lock(A)

Resolve the Deadlock

One transaction must be aborted/rolled back.

For example, abort T2.

T2 → ABORT

↓

Release X-lock(B)

↓

T1 obtains X-lock(B)

↓

T1 completes

↓

COMMIT T1

Restart T2

After T1 finishes:

T1:

X-lock(A)

X-lock(B)

Read/Write A

Read/Write B

COMMIT

Unlock A

Unlock B

Now T2 can restart:

T2:

X-lock(B)

X-lock(A)

Read/Write B

Read/Write A

COMMIT

Unlock B

Unlock A

The deadlock is removed.

Correct 2PL Execution

The corrected execution can be:

T1:

X-lock(A)

X-lock(B)

Read(A)

Write(A)

Read(B)

Write(B)

COMMIT

Unlock(A)

Unlock(B)

T2:

X-lock(B)

X-lock(A)

Read(B)

Write(B)

Read(A)

Write(A)

COMMIT

Unlock(B)

Unlock(A)

How to Avoid the Deadlock

A simple way to prevent this particular deadlock is to make all transactions request locks in the same order.

For example:

Always lock A first

Then lock B

So both transactions follow:

X-lock(A)

↓

X-lock(B)

↓

Perform operations

↓

Commit

↓

Release locks

Then T2 waits for T1 to release A instead of creating a circular wait.

Final Answer

T1 holds A → waits for B

T2 holds B → waits for A

↓

Deadlock

↓

Wait-for graph

T1 → T2

T2 → T1

↓

Cycle

↓

Abort T2

↓

Release B

↓

T1 completes

↓

Restart T2

Result

Basic Two-Phase Locking was successfully demonstrated. A deadlock occurred because T1 and T2 were waiting for each other's locks. The deadlock was identified using a wait-for graph and resolved by aborting T2, releasing its locks, allowing T1 to complete, and then restarting T2.

6. Apply the Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes on a given set of transactions requesting locks.

Ans:

Given Transactions

Consider three transactions with the following timestamps:

Transaction	Timestamp	Age
T1	10	Older
T2	20	Younger
T3	30	Youngest

Rule

The transaction with the smaller timestamp is older.

Therefore:

T1 (10) → Oldest

T2 (20) → Younger

T3 (30) → Youngest

Given Lock Requests

Assume the following situation:

T1 holds X-lock(A)

T2 requests X-lock(A)

T3 holds X-lock(B)

T1 requests X-lock(B)

So:

Transaction	Holds	Requests
T1	X-lock(A)	X-lock(B)
T2	—	X-lock(A)
T3	X-lock(B)	—

Apply Wait-Die Scheme

Wait-Die Rules

In Wait-Die:

- Older transaction requests a lock held by younger → WAIT
- Younger transaction requests a lock held by older → ABORT (DIE)

Case 1: T2 requests A

A is held by T1.

T2 timestamp = 20

T1 timestamp = 10

T1 is older than T2.

Therefore:

T2 (younger)

↓

Requests lock held by

↓

T1 (older)

↓

T2 ABORTS

So:

T2 → DIE/ABORT

Case 2: T1 requests B

B is held by T3.

T1 timestamp = 10

T3 timestamp = 30

T1 is older than T3.

Therefore:

T1 (older)

↓

Requests lock held by

↓

T3 (younger)

↓

T1 WAITS

So:

T1 → WAIT

Wait-Die Result

Request	Requesting Transaction	Lock Holder	Result
T2 requests A	T2 (younger)	T1 (older)	T2 aborts
T1 requests B	T1 (older)	T3 (younger)	T1 waits

Therefore:

Wait-Die:

Older → WAIT

Younger → DIE/ABORT

Apply Wound-Wait Scheme

Wound-Wait Rules

In Wound-Wait:

- Older transaction requests a lock held by younger → Younger is ABORTED
- Younger transaction requests a lock held by older → WAIT

Case 1: T2 requests A

A is held by T1.

T2 = 20 → Younger

T1 = 10 → Older

T2 is younger and requests a lock held by older T1.

Therefore:

T2 (younger)

↓

Requests A held by

↓

T1 (older)

↓

T2 WAITS

So:

T2 → WAIT

T1 requests B

B is held by T3.

T1 = 10 → Older

T3 = 30 → Younger

T1 is older and requests a lock held by younger T3.

Therefore:

T1 (older)

↓

Requests B held by

↓

T3 (younger)

↓

T3 ABORTS

So:

T3 → WOUNDED/ABORTED

Wound-Wait Result

Request	**Requesting Transaction**	**Lock Holder**	**Result**
T2 requests A	T2 (younger)	T1 (older)	T2 waits
T1 requests B	T1 (older)	T3 (younger)	T3 aborts

Therefore:

Wound-Wait:

Older → WOUND/ABORT Younger

Younger → WAIT

Compare Both Schemes

Situation	**Wait-Die**	**Wound-Wait**
Older requests younger's lock	Older waits	Younger aborts
Younger requests older's lock	Younger aborts	Younger waits
T2 → T1	T2 aborts	T2 waits
T1 → T3	T1 waits	T3 aborts

Easy Memory Trick

Wait-Die

OLDER → WAIT

YOUNGER → DIE

Wound-Wait

OLDER → WOUND

YOUNGER → WAIT

Final Result

Wait-Die

T2 requests A held by T1

↓

T2 is younger

↓

T2 ABORTS

T1 requests B held by T3

↓

T1 is older

↓

T1 WAITS

Wound-Wait

T2 requests A held by T1

↓

T2 is younger

↓

T2 WAITS

T1 requests B held by T3

↓

T1 is older

↓

T3 ABORTS

Conclusion

Wait-Die and Wound-Wait prevent deadlocks by using transaction timestamps. Wait-Die allows older transactions to wait and aborts younger transactions, whereas Wound-Wait allows younger transactions to wait and aborts younger transactions when an older transaction requests their lock.

7. Given transaction logs with checkpoints, apply the Immediate Update recovery protocol to recover the database after a system failure.

Understand Immediate Update

In Immediate Update, changes made by a transaction can be written to the database **before the transaction commits**.

Therefore, after a crash:

- **Committed transactions → REDO**
- **Uncommitted transactions → UNDO**

So:

Immediate Update = UNDO + REDO

Given Transaction Log

Consider the following transaction log:

Log No.	Log Record
1	<START T1>
2	<T1, A, 100, 150>
3	<START T2>
4	<T2, B, 200, 250>
5	<COMMIT T1>
6	<CHECKPOINT>
7	<START T3>
8	<T3, C, 300, 350>
9	<COMMIT T2>
10	SYSTEM CRASH

The format:

<T1, A, 100, 150>

means:

Transaction = T1

Data item = A

Old value = 100

New value = 150

Identify the Checkpoint

The checkpoint occurs at:

<CHECKPOINT>

This tells the recovery system that earlier committed transactions may already have been safely processed.

The checkpoint helps reduce the amount of log that needs to be examined.

Identify Transaction Status at Failure

At the time of the crash:

T1

T1 → COMMITTED

because:

<COMMIT T1>

appears in the log.

T2

T2 → COMMITTED

because:

<COMMIT T2>

appears before the crash.

T3

T3 → UNCOMMITTED

because there is no:

<COMMIT T3>

before the crash.

Therefore:

Transaction	**Status**
T1	Committed
T2	Committed
T3	Uncommitted

Identify Transactions for REDO

In Immediate Update:

Committed → REDO

Therefore:

T1 → REDO

T2 → REDO

REDO T1

Log record:

<T1, A, 100, 150>

T1 committed.

Therefore, apply the new value:

A = 150

So:

Old value = 100

New value = 150

REDO operation

A: 100 → 150

REDO T2

Log record:

<T2, B, 200, 250>

T2 committed.

Therefore:

B = 250

REDO operation

B: 200 → 250

Identify Transaction for UNDO

In Immediate Update:

Uncommitted → UNDO

T3 did not commit.

Therefore:

T3 → UNDO

UNDO T3

Log record:

<T3, C, 300, 350>

The old value is:

300

The new value is:

350

Since T3 did not commit, restore the old value:

C = 300

Therefore:

C: 350 → 300

Recovery Table

Transaction	Status	Recovery Action	Final Value
T1	Committed	REDO	A = 150
T2	Committed	REDO	B = 250
T3	Uncommitted	UNDO	C = 300

Final Database State

After recovery:

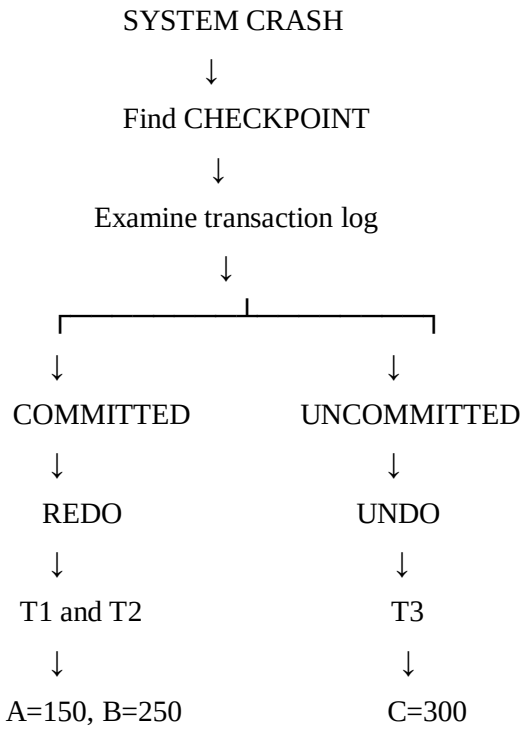
A = 150

B = 250

C = 300

The effects of committed transactions are preserved, while the effect of the uncommitted transaction is removed.

Recovery Flow



Conclusion

Using the Immediate Update recovery protocol, T1 and T2 are REDONE because they committed before the system failure. T3 is UNDONE because it was uncommitted at the time of failure. Therefore, the final recovered database contains A = 150, B = 250, and C = 300.

8. Apply Deferred Update (NO-UNDO/REDO) recovery technique on a given log file with committed and uncommitted transactions.

Understand Deferred Update

In Deferred Update, changes made by a transaction are not written to the database until the transaction commits.

Therefore, after a system crash:

- Committed transactions → REDO
- Uncommitted transactions → Ignore
- UNDO is not required

Given Transaction Log

Consider the following log:

Log No.	Log Record
1	<START T1>
2	<T1, A, 100, 150>
3	<START T2>
4	<T2, B, 200, 250>
5	<COMMIT T1>
6	<START T3>
7	<T3, C, 300, 350>
8	<COMMIT T2>
9	<START T4>
10	<T4, D, 400, 450>
11	SYSTEM CRASH

Identify Transaction Status

Look for a COMMIT record before the crash.

T1

<COMMIT T1>

Therefore:

T1 = Committed

T2

<COMMIT T2>

Therefore:

T2 = Committed

T3

There is no:

<COMMIT T3>

Therefore:

T3 = Uncommitted

T4

There is no:

<COMMIT T4>

Therefore:

T4 = Uncommitted

Prepare the Transaction Status Table

Transaction	Status	Recovery Action
T1	Committed	REDO
T2	Committed	REDO
T3	Uncommitted	Ignore
T4	Uncommitted	Ignore

REDO T1

Log record:

<T1, A, 100, 150>

T1 committed.

Therefore, apply the new value:

A = 150

So:

A: 100 → 150

REDO T2

Log record:

<T2, B, 200, 250>

T2 committed.

Therefore:

B = 250

So:

B: 200 → 250

Ignore T3

Log record:

<T3, C, 300, 350>

T3 did not commit.

Under Deferred Update, its changes were not written to the database.

Therefore:

T3 → IGNORE

No UNDO operation is performed.

Ignore T4

Log record:

<T4, D, 400, 450>

T4 also did not commit.

Therefore:

T4 → IGNORE

No UNDO operation is required.

Recovery Table

Transaction	**Operation**	**Status**	**Recovery**
T1	A: 100 → 150	Committed	REDO
T2	B: 200 → 250	Committed	REDO
T3	C: 300 → 350	Uncommitted	Ignore
T4	D: 400 → 450	Uncommitted	Ignore

Final Database State

After recovery:

A = 150

B = 250

T3 and T4 changes are not present because they were uncommitted.

Therefore:

C = 300

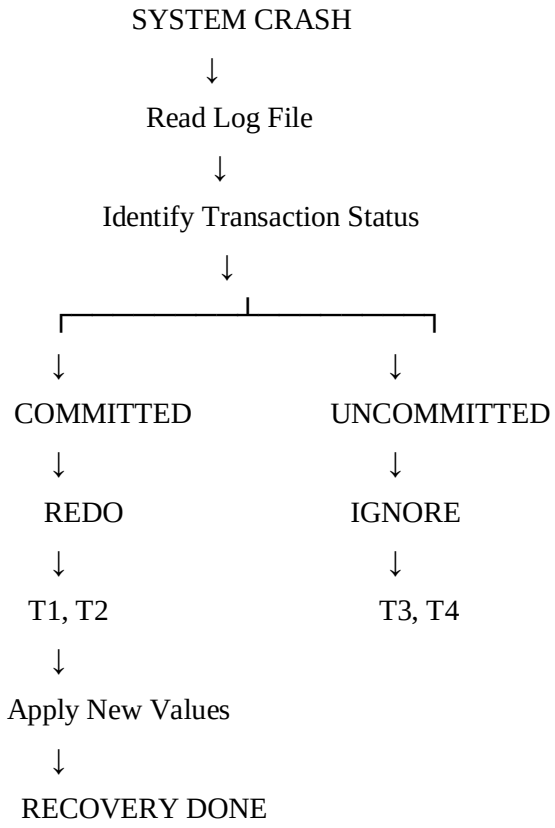
D = 400

if their original values were 300 and 400.

Final State

Data Item	Final Value
A	150
B	250
C	300
D	400

Recovery Flow



Conclusion

Using Deferred Update recovery, T1 and T2 are committed transactions, so their updates are REDONE. T3 and T4 are uncommitted transactions, so their updates are ignored. No UNDO operation is required because deferred update writes changes to the database only after a transaction commits.

9. Design a concurrency control mechanism using strict 2PL for a banking system where T1 transfers funds and T2 reads balance simultaneously.

Define Transaction T1

T1 transfers ₹2,000 from Account A to Account B.

The transaction is:

T1:

Lock A

Lock B

↓

Read A

A = A - 2000

Write A

↓

Read B

B = B + 2000

Write B

↓

COMMIT

↓

Unlock A and B

```
mysql> CREATE DATABASE banking_db;
Query OK, 1 row affected (0.06 sec)

mysql> USE banking_db;
Database changed
mysql> CREATE TABLE account ( account_id INT PRIMARY KEY, account_name VARCHAR(50), balance DECIMAL(10,2) );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO account VALUES (101, 'Account A', 10000.00), (102, 'Account B', 5000.00);
Query OK, 2 rows affected (0.05 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM account;
+------------+--------------+----------+
| account_id | account_name | balance  |
+------------+--------------+----------+
|        101 | Account A    | 10000.00 |
|        102 | Account B    |  5000.00 |
+------------+--------------+----------+
2 rows in set (0.00 sec)
```

Implement T1 in MySQL

Open **MySQL Session 1**.

Execute:

USE banking_db;

START TRANSACTION;

SELECT *
FROM account
WHERE account_id IN (101,102)
FOR UPDATE;
FOR UPDATE obtains **exclusive row locks** on the selected accounts.
Now transfer the money:
UPDATE account
SET balance = balance - 2000
WHERE account_id = 101;
Then:
UPDATE account
SET balance = balance + 2000
WHERE account_id = 102;
At this point T1 has changed the balances but **has not committed**.

Define Transaction T2
T2 wants to read the balances while T1 is transferring money.
T2 should use shared/read locks if we want to explicitly demonstrate strict locking.
Open **MySQL Session 2** and execute:
USE banking_db;
START TRANSACTION;
SELECT *
FROM account
WHERE account_id IN (101,102)
FOR SHARE;
Because T1 is still holding exclusive locks, T2 has to **wait** until T1 commits or rolls back.
T1 holds X-lock
↓
T2 requests S-lock
↓
T2 waits

Commit T1

Go back to **MySQL Session 1**.

Execute:

COMMIT;

Now T1 releases its locks.

T1

↓

COMMIT

↓

Release locks

↓

T2 can continue

T2 Reads the Updated Balance

T2 can now obtain its shared locks and read the balances.

SELECT *

FROM account

WHERE account_id IN (101,102);

Expected result:

Account ID	Balance
101	₹8,000
102	₹7,000

Then finish T2:

COMMIT;

What Happens Without Strict 2PL?

Suppose T2 reads while T1 has changed Account A but has not yet changed Account B:

T1: A = A - 2000

↓

T2: Reads A = 8000

↓

T2: Reads B = 5000

↓

T1: B = B + 2000

T2 sees:

A = 8000

B = 5000

Total = ₹13,000.

But the correct total before the transfer was:

10000 + 5000 = ₹15000

Therefore T2 has read an **inconsistent state**.

Strict 2PL prevents this.

Strict 2PL Mechanism

The mechanism is:

T1

X-lock(A)

X-lock(B)

↓

Update A

Update B

↓

COMMIT

↓

Unlock A

Unlock B

T2

Request S-lock(A)

Request S-lock(B)

↓

WAIT

↓

T1 COMMIT

↓

Acquire locks

↓

Read balances

↓

COMMIT

Final State

Initial:

Account A = ₹10,000

Account B = ₹5,000

T1 transfers ₹2,000:

Account A = ₹8,000

Account B = ₹7,000

T2 reads only after T1 commits:

Account A = ₹8,000

Account B = ₹7,000

Total remains:

₹8,000 + ₹7,000 = ₹15,000

So the banking system remains consistent.

Result

A Strict 2PL concurrency control mechanism was successfully designed for the banking system. T1 obtains exclusive locks on both accounts while transferring funds and holds them until commit. T2 waits until T1 commits before reading the balances. Thus, dirty reads and inconsistent intermediate balances are prevented, ensuring database consistency and reliability.

10. Write pseudocode for query optimization using heuristics: push down selection, project early, and order joins by smaller intermediate results.

Objective:

To optimize a query by reducing unnecessary data before performing joins.

Step 1: Start with the Query

Input: SQL Query

Step 2: Convert Query

Convert SQL query into a query tree.

Step 3: Push Down Selection

Move selection conditions closer to the base tables.

Example:

Before:

JOIN → SELECT

After:

SELECT → JOIN

This reduces the number of rows before joining.

Step 4: Project Early

Keep only the required columns from each table.

Example:

STUDENT → Keep Student_ID, Name, Dept_ID

This reduces the amount of data processed.

Step 5: Order Joins

Estimate the size of intermediate results.

Perform the join producing the smaller result first.

Step 6: Generate Optimized Query Tree

Selection

↓

Projection

↓

Smallest Join

↓

Next Join

↓

Final Result

Pseudocode

Algorithm Optimize_Query(Query)

1. Parse Query
2. Create Query Tree
3. For each selection condition:
    Push selection toward base table
4. For each table:
    Keep only required attributes
    Push projection toward base table
5. Estimate intermediate result sizes
6. Order joins:
    Perform smaller-result joins first
7. Generate optimized query tree
8. Return optimized query

End Algorithm

Result

Heuristic query optimization reduces intermediate data, disk I/O, memory usage, and query execution time by pushing selections and projections early and performing smaller joins first.